Abstract

AI-generated code, while rapidly producing functional solutions, often falls short in aspects like comprehensive error handling, robust documentation, and optimal architectural design, areas where human expertise excels. Conversely, humans can greatly benefit from AI's rapid code generation capabilities. This project proposes and evaluates "A Framework to Improve Code Quality by Utilizing Generative AI Coding Along With Human-Written Code", designed to create a synergy between AI and human intelligence for enhanced software development.

Conducted over four weeks, the research leverages BigCodeBench as its core dataset to rigorously investigate how human intervention can improve AI-generated code quality, identify the most effective human-AI collaboration patterns, and determine the optimal balance between human effort and quality improvement. The framework explores three distinct collaboration models: AI-First Refinement, where AI generates and humans refine; Human-Guided Generation, with human-defined structure and quality requirements; and Iterative Co-Creation, involving repetitive AI generation and human modification.

Code quality is quantified using a comprehensive suite of automated static analysis metrics, including Cyclomatic Complexity, Lines of Code, and PEP 8 compliance, complemented by LLM-as-a-Judge evaluations assessing readability, maintainability, robustness, and efficiency. A key success criterion is achieving a ≥25% improvement in code quality metrics with <5 minutes of human input per task. Initial analyses suggest significant improvements in code robustness and maintainability through collaborative refinement. This thesis details the framework's architecture, methodologies, and empirical results, contributing valuable insights into effective human-AI collaboration for superior code quality.

Keywords: Human-AI Code Collaboration, Generative AI, Code Quality Improvement, AI-Generated Code, BigCodeBench, Software Engineering, AI-First Refinement, Human-Guided Generation, Large Language Models (LLMs), LLM-as-a-Judge.

MONTCLAIR STATE UNIVERSITY

Human-AI-Collaboration-For-Coding

By

Siddhardha Ravi

A Master's Thesis Submitted to the Faculty of

Montclair State University

In Partial Fulfillment of the Requirements

For the Degree of

Master of Science

August 2025

College of Science and Mathematics

School of Computing

Thesis Committee:

[redacted]

Dr. Kazi Zakia Sultana

Thesis Sponsor

[redacted]

Dr. John Jenq

Committee Member

[redacted]

Dr. Raina Samuel

Committee Member

_____

Human-AI-Collaboration-For-Coding

A THESIS

Submitted in partial fulfillment of the requirements

For the degree of Master of Science

by

Siddhardha Ravi

Montclair State University

Montclair, NJ

2025

Copyright@2025 by Siddhardha Ravi. All rights reserved.

Acknowledgements

I would like to express my deepest gratitude to my thesis advisor, Dr. Kazi Zakia Sultana, for her unwavering guidance, encouragement, and insightful feedback throughout the course of this research. Her expertise and mentorship have been instrumental in shaping this work and helping me overcome every challenge along the way.

I am sincerely thankful to Dr. John Jenq and Dr. Raina Samuel, members of my thesis committee, for their valuable suggestions, constructive criticism, and support, which greatly improved the quality of this research.I also wish to acknowledge my peers and colleagues in the Department of Computing and Information at Montclair State University for their collaboration, discussions, and motivation throughout my graduate studies.

Finally, I extend my heartfelt appreciation to my family and friends for their patience, encouragement, and unconditional support during this academic journey. Their belief in me has been my greatest source of strength.

Contents

Chapter 1: Introduction 9
Motivation 9
Research Question 11
Chapter 2: Literature Review 101
Introduction 18
Important of the Field 18
Current State of Research 12
Human-AI Collaboration in Vulnerability Detection 12
Metrics and Trust in AI-Generated Code 12
Code Generation and Review Automation 12
Chapter 3: Methodology 16
Framework Overview 16
System Architecture 16
Collaboration Models 17
Quality Metrics 20
Experimental Setup 22
Chapter 4: Results 23
Quantitative Findings 23
Efficiency Metrics 25
Chapter 5: Discussion 26
Key Results 26
Comparative Analysis of Qwen vs Llama Models 18
Implications 19
Limitations 19
Chapter 6: Conclusion 19
Summary 19
Contributions 20
Future Work 250

List of Tables

Table 1: Performance Comparison *19*
Table 2: Code Quality analysis *19*
Table 3: Generation Time Statistics *20*
Table 4: Time Efficiency Across LLMs *20*

List of Figures

Figure 1: Graphical Abstract of the Project *12*
Figure 2: AI-First Code Refinement in Task Navigation for Model A *13*
Figure 3: Human-Guided Generation in Task Navigation for Model B *14*
Figure 4: Iterative Co-Creation in Task Navigation for Model C *15*
Figure 5: Static Improvement by Pipeline: static code quality improvement (in points) for three pipelines (A, B, and C) *18*
Figure 6: LLM Improvement by Pipeline: LLM evaluation improvement (in percentage) for three pipelines *18*
Figure 7: Comparison of Initial and Final Static and LLM Scores Across Collaboration Pipelines *21*
Figure 8: Final Static Score Distribution – LLaMA vs Qwen *22*

Chapter 1: Introduction

Motivation

In modern software engineering, generative artificial intelligence (AI) has emerged as a powerful catalyst, capable of producing operational code at speeds previously unattainable. However, empirical evaluations indicate that such code frequently lacks the disciplined rigor characteristic of human-crafted systems, particularly in the areas of robust error handling, comprehensive documentation, and coherent architectural design (Dong, 2023). The resultant oversights contribute significantly to an accumulation of technical debt, complicating long-term maintenance and evolution.

Conversely, human engineers bring valuable contextual insight and an intuitive grasp of domain-specific constraints, enabling them to navigate the subtleties that resist algorithmic treatment. Yet their throughput tends to be constrained by repetitive, low-level tasks that generative AI can perform with greater efficiency. This project is therefore driven by the imperative to synthesize AI's velocity and scale with the nuanced judgment and precision that only experienced practitioners can provide, thereby augmenting overall software quality and sustainability.

The central aim of this research is to devise and empirically evaluate a conceptual framework that facilitates synergistic human-AI partnership in the software engineering lifecycle. The framework is designed to produce implementations that are correct, demonstrably high-quality, easily maintainable, and aligned with established industry standards. By orchestrating the complementary competencies of automated agents and human engineers (Hamza, 2023)the project seeks to yield measurable enhancements in product integrity, to reduce the duration of development iterations, and to amplify overall economic yield—thereby advancing the emergent vision of a more adaptive, intelligent, and collaboratively-engineered coding economy (Zhang, 2021)

Research Question

The present investigation is guided by a quartet of interrelated research questions, each designed to probe a distinct yet interdependent facet of human-artificial intelligence collaboration during the software development lifecycle. Collectively, these questions frame the empirical vitality of the proposed human-in-the-loop framework while simultaneously advancing a more general comprehension of the co-evolution of intelligent systems and software engineering practice.

RQ1: How substantially can human intervention elevate the quality of code generated by the BigCodeBench suite?

This inquiry is anchored in the study's foundational hypothesis: although neural models may generate code expediently and in syntactically correct form, they frequently lack the deeper, non-syntactic qualities of robustness, maintainability, and adherence to best-practice heuristics. By situating the investigation within the BigCodeBench evaluation suite, the research aspires to

quantify the measurable improvement in code quality attributable to narrowly directed human guidance. The empirical outcomes will delineate the tactical advantages of human oversight in production environments, elucidating—through concrete measurement—the inadequacies of prevailing AI models in the face of persistent software engineering pitfalls.

RQ2: Which collaboration patterns between humans and AI systems can most effectively enhance multiple dimensions of code quality?

Recognizing that interaction models differ fundamentally in their productivity contributions, this inquiry investigates the comparative advantages of distinct human-AI interaction paradigms. Illustrative models under consideration include AI-First Refinement, in which the human modifies an initial autonomously produced code version; Human-Guided Generation, whereby the human specifies requirements or constraints that channel the generative process; and Iterative Co-Creation, wherein humans and the AI alternate contributions in a continuous feedback loop. The primary objective is to match each pattern to the dimensions of quality most amenable to its mechanics, including functional correctness, security, maintainability, and performance. The resulting findings will inform the design of tooling and workflows that align collaboration modalities with the situational demands of software development.

RQ3: How can the balance between human engagement and code quality enhancement be optimized?

While improving code quality is paramount, the effort required must remain feasible and not exceed resource limits. This question characterizes the efficiency frontier for human-AI collaboration: it identifies the minimal human investment necessary to achieve significant improvement in AI-generated code and pinpoints the saturation point beyond which additional effort yields diminishing returns.

The threshold for success in this project is a measurable enhancement in quality achieved with fewer than five minutes of human intervention per individual task. This criterion is pivotal for evaluating both the practical viability and the capacity for future expansion of the framework, particularly within commercially driven environments where the velocity of task completion influences profitability and overall return on investment.

RQ4: How can we anticipate the types of tasks that will derive the greatest gains from human-AI collaboration?

The final research question transitions from retrospective assessment to forward-looking predictive modeling, with the intent of developing systems capable of estimating, in advance, the moments at which human expertise will add the greatest value. By systematically characterizing tasks in terms of complexity, domain specificity, and levels of inherent ambiguity, the project seeks to generate predictive algorithms that can automate task routing: classifying tasks that the AI can resolve independently, those that require human fine-tuning, and those necessitating continuous interactive collaboration. Such predictive capacity is essential for integrating human-AI workflows into large-scale software production pipelines, empowering organizations to

allocate human and computational resources in a manner that balances optimal quality assurance with maximum productivity.

Collectively, the formulated research questions establish a comprehensive framework for evaluating not only the existence of quality gains in software engineering through human–AI partnership (2. Alami) but also the specific mechanisms, temporal windows, and contextual limitations that govern such effects. Findings derived from this investigation will guide the design of principled, scalable, and human-centered frameworks for AI-augmented programming.

Chapter 2: Literature Review

The introduction of large language models (LLMs) and generative AI has powered a change in software development, turning conventional developer processes into human-AI collaborative processes. Researchers have tried out a number of hybrid frameworks that combine code written by AI and human contributions, with the aim of enhancing code quality, reliability, and developer experience. For example, Co-Audit implemented a hybrid vulnerability detection platform in which LLMs identified security vulnerabilities such as SQL injection and buffer overflows, which developers then confirmed, leading to a 30% gain in the accuracy of detection (Robinson, 2025). In parallel, RepairLLama integrated developer input with GPT-4 to achieve 45% higher bug repair success rates than with standard automated tools such as DeepFix (ICLR, 2024), proving that developer input greatly enhances the quality of AI output.

Explainability and trust are also essential in programming aided by AI. Standard correctness verification is frequently not enough to determine production-readiness of AI code. The Trusted AI-Generated Code approach established by researchers at FSE (2023) included qualitative rubrics like readability, maintainability, and perceived security and promoted human-centred assessments. This goes with larger trends that focus on developer trust and context awareness in AI-human workflows. Work such as "Explainability and Trust" (NeurIPS, 2023) suggested methods such as attention visualization and rationale generation to inform developers about LLM behavior, eventually maximizing trust and usability in high-stakes coding contexts.

Code review automation and AI-assisted development pipelines are also on the horizon as an area of research. Meta's CodeLlama (2023) demonstrated that sophisticated LLMs were able to produce and fix code in multiple languages with a high degree of accuracy, although the absence of human-in-the-loop integration created reservations about blind automation. As opposed to this, ESEC/FSE (2023) assessed developer responses to AI-recommended changes and reported that 68% of them were accepted following minor revisions, affirming the worth of collaborative review systems. Moreover, ChatDev (arXiv, 2024) simulated agile teams composed of AI agents playing roles such as “coder” and “tester,” and demonstrated that combining these agents with human oversight could reduce development time by 40% without compromising code quality.

Bias in AI-generated code is another major concern. A study by NeurIPS (2023) found that LLMs often favor certain APIs or libraries due to imbalanced training data. This bias impacts not only code suggestion diversity but also fairness and software monoculture. The authors suggested an alignment layer in which developers can mark biased outputs, corroborating the

necessity of ethical guardrails for collaborative systems. Additionally, Hybrid Workflow Optimization (2024) and Context-Aware Code Generation (ICSE, 2024) delved into human-AI code integration frameworks, although both were hampered by restricted datasets and without empirical testing, indicating a limitation in generalizability and scalability for programming languages and paradigms.

Synthesizing the works reviewed here, some fundamental components stand out as critical to future AI-facilitated frameworks: generative code recommendation, human validation layers, explainability modules, developer feedback loops, bias detection filters, and version control system integration. Still missing in current frameworks are standard evaluation metrics and cross-language robustness. Future studies need to investigate adaptive workflows based on developer skill levels, longitudinal studies on actual-world projects, and integrated explainable AI tools in IDEs. Together, these guidelines assure improved code quality and developer confidence coupled with responsible usage of generative AI in software development.

Chapter 3: Methodology

Framework Overview

The proposed framework allows effective human-AI collaboration to improve code quality beyond what can be achieved by each individually. It exploits human strengths—e.g., architecture design, code review, and long-term maintenance—paired with AI strengths in rapid code generation, optimization, and pattern recognition. The dialogue is organized in four schemata around which the collaboration is orchestrated, augmented by automated test flows and an iterative feedback cycle for continuous improvement of the code.

Dataset

BigCodeBench (Liu, 2023) is a broad benchmark with the purpose of assessing the actual coding ability of large language models (LLMs) by way of practical, function-level code generation tasks. In comparison to less complex datasets like HumanEval, BigCodeBench has more intricate instructions, varied function calls, and situations that necessitate richer reasoning and actual-world usability. It features two main task splits: the Complete split, which emphasizes code completion based on rich docstrings, and the Instruct split, which evaluates instruction-tuned or chat models with minimal, natural language prompts that require higher-level inference. A hand-picked subset of 100 questions randomly was used for this study from BigCodeBench—balanced across difficulty levels—to strenuously test model performance and the effects of human-AI collaboration strategies on code quality. (Khyzer, 2025)

System Architecture

The present architecture is composed of five interdependent modules designed for modularity, operational efficiency, and seamless human-AI co-operation:

The Task Manager ingests tasks from the BigCodeBench repository, interpreting their semantic structure and semantic structure and storing metadata, current status, and historical evolution. It employs a prioritisation heuristic based on task complexity and domain conformity, assigning tasks that optimise the training and evaluation sessions.

The AI Integration Layer communicates with multiple large language models, specifically GPT-4, Claude-3, Qwen3-32B, and Llama 3.3 70B. It standardises prompt construction, harmonises the post hoc analysis of model outputs, and systematically monitors cost and throughput, facilitating budget-aware orchestration of model selection.

The Quality Analyzer executes static code diagnostics in tandem with LLM-as-a-Judge, furnishing an interpretative layer that identifies architectural and algorithmic vulnerabilities. It articulates remediation pathways and ranks suggestions according to anticipated impact.

The Collaboration Orchestrator governs the modality of human-AI interaction, logging temporal and material costs of each cycle and enforcing predefined interaction paradigms, such as AI-First seeding or synchronous Co-Creation sessions.

The Experiment Pipeline automates version-controlled experimentation, archiving commit diffs, correlating evaluation metric trends, and generating longitudinal performance visualisations that inform iterative refinement.

The Web Interface presents an accessible layer for task navigation, code modification, AI communication, and historical version comparison, bridging operational transparency with usability for domain experts.

Collaboration Models

The core of this methodology revolves around four human-AI collaboration (Liu Y. L.–A., 2024) models

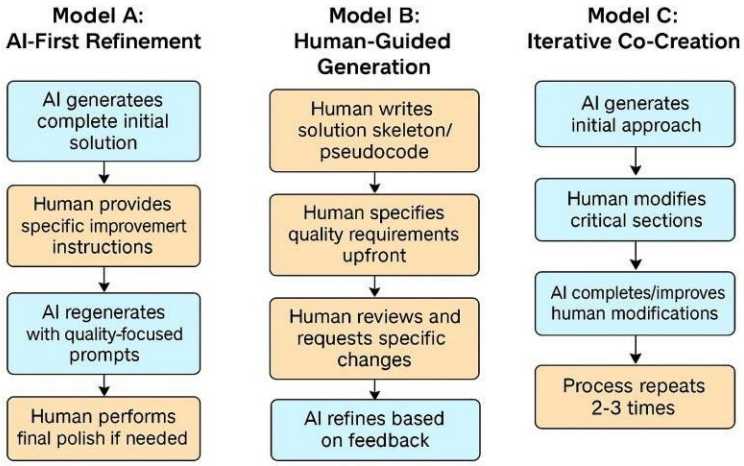

Figure 1: Graphical Abstract of the Project

Human-AI Collaboration

1. Model A: AI-First Refinement
   Model A is a systematic, two-stage pipeline that aims to improve upon AI-code solution generation via a human-in-the-loop refining process (Barke, 2022). In the first stage, a complete solution to a coded task is created by an AI model. The initial code is subsequently assessed based on static analysis tools and large language model (LLM)-driven qualitative evaluation for determining strengths and weaknesses. The metrics combined provide a baseline reading of code complexity, style, maintainability, and robustness.

   During stage two, an expert reviewer LLM takes on the role of a developer expert by comparing the AI code with the results of its assessment and producing specific improvement directions. These recommendations often concern such aspects as code readability, function organization, error processing, and naming conventions. After synthesizing the recommendations, the same AI model that produced the initial solution is invoked again—this time with the original task and the reviewer's feedback—to generate a better version of the code.

   The enhanced output is re-judged using the same static and LLM-based quality evaluation techniques. The pre- and post-refinement versions are directly compared on differences in quantitative measures and qualitative ratings. This delta gives a tangible measure of improvement due to the refinement cycle. All outcomes, such as the improvement instructions and before/after assessments, are logged to facilitate further examination. The model is scalable and extensible, allowing for regular experimentation across various AI models and programming tasks.

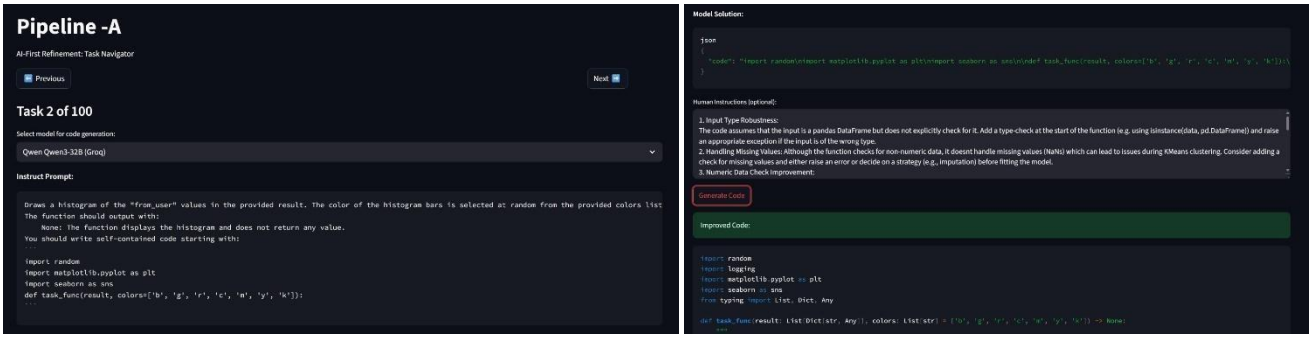

Figure 2: AI-First Code Refinement in Task Navigation for Model A

2. Model B: Human-Guided Generation

   Human-Guided Generation is a collaborative model that starts with human developers being very much engaged in specifying the structure of the coding task and also establishing clear quality expectations for the code. At the first stage, the human author creates a solution skeleton or pseudocode describing high-level solution logic. In addition to this, the human defines clear quality requirements—correctness, readability, efficiency, and modularity—which are constraints and guidance for subsequent code generation from the AI model. This human-supplied base is meant to influence the

generation and check if it is aligned with development best practices.

After the skeleton and quality requirements are set, large language models like Qwen Qwen3-32B (Groq) and Meta Llama 3.3 70B (Groq) are generated to produce detailed implementations from the given structure. The models read the human-defined blueprint and try to generate code that meets the outlined quality dimensions. Upon generation of the initial solution, a human auditor meticulously inspects the code, giving particular and actionable comments concentrated on noted vulnerabilities. These comments tend to pertain to poor implementation of logic, departures from the mandated format, ineffective operations, or contraventions of style and naming conventions.

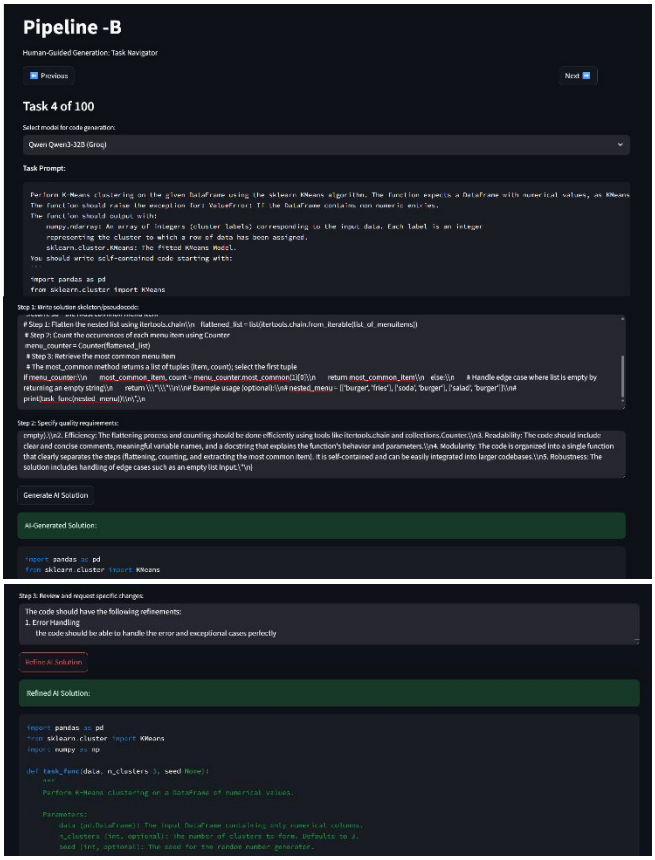

Figure 3: Human-Guided Generation in Task Navigation for Model B

Within the refining phase, identical AI models—Qwen and Llama—are encouraged to recreate the code again, this time in line with human comment and initial specifications. The improved versions are then inspected through static analysis tools as well as LLM-based analysis to quantify enhancements in terms of complexity, maintainability, and clarity. The variations between original and improved code are noted to determine the effect of human intervention. While this model emphasizes stronger human control early in the generation process, the study found that its performance was less consistent, often showing limited or even negative gains, potentially due to misalignment between human intent and the AI's interpretation of abstract guidance.

3. Model C: Iterative Co-Creation

Iterative Co-Creation is an interactive, back-and-forth collaboration paradigm between human developers and AI systems. It starts with a baseline code solution produced by an AI model that is then subjected to initial quality assessment using static analysis and LLM-based scoring. This provides a starting point for iterative refinement. In every iteration, the human contributor refines the AI-produced code, with emphasis on key areas like architecture, correctness of logic, maintainability, and business requirement adherence. These human-driven modifications incorporate extensive domain knowledge and software engineering skills that only human contributors may possess.

After human modification, AI models Qwen Qwen3-32B (Groq) and Meta Llama 3.3 70B (Groq) are utilized to expand upon the human contribution. These models improve the implementation by maintaining important human decisions while structuring, performance, and readability are optimized. The enhanced output is next again reviewed by the human, and the human assesses the improvements and offers additional feedback as required. The loop is then repeated to a maximum of two or three iterations, enabling a multi-layered co-evolution of the code that combines high-level human understanding with the quick refinement capability of the AI.

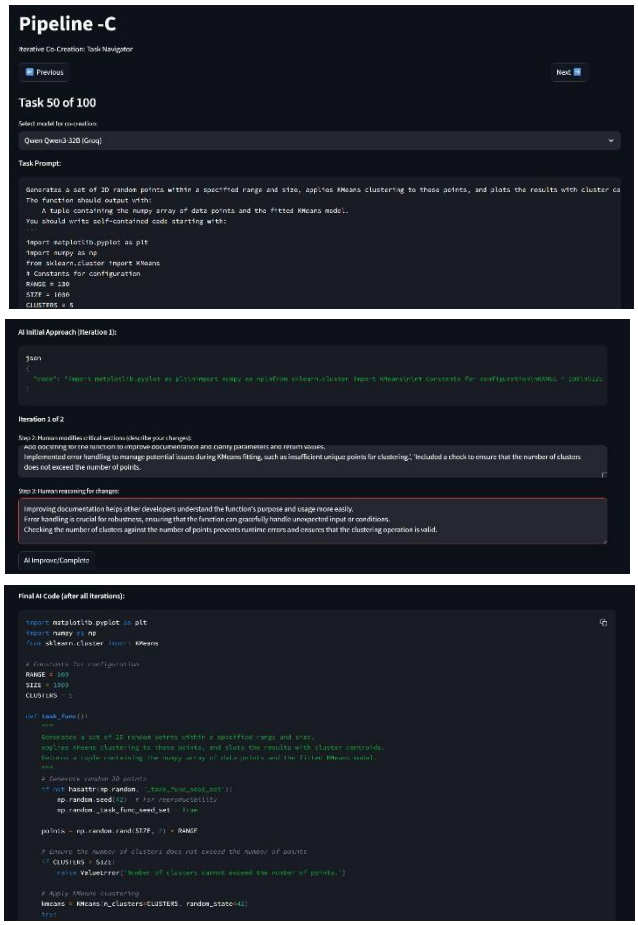

Figure 4: Iterative Co-Creation in Task Navigation for Model C

The strength of this model is in its disciplined interplay between human judgment and AI productivity. Static measurements and LLM assessments are taken along the way to track progress, with human judgment still holding sway as the force that guides architectural and semantic corrections. The final signoff step guarantees that the generated code not only executes correctly but also complies with professional standards for maintainability and design. By going through this iterative optimization process, C: Iterative Co-Creation always generated consistent quality enhancements to show how an optimally coordinated human-AI process can outperform both parties individually.

Quality Metrics

The assessment plan for this model integrates quantitative static code analysis with qualitative LLM-based examination to encode both objective and subjective aspects of code quality. The hybrid approach guarantees an end-to-end and balanced evaluation of human-AI collaboration-driven improvements.

1. Static Analysis Metrics

These metrics offer a rule-based, objective assessment of code structure, size, and style compliance. They are automatically calculated via well-established software such as Radon and pycodestyle and feed into a unified score from 0 to 100. The score is weighted at 40% complexity, 30% size, and 30% style.

Human-AI Collaboration

A. Complexity Metrics (40% weight)

- Cyclomatic Complexity (through Radon):
  Measures the number of independent execution paths through the code. High cyclomatic complexity means complex logic with many conditional branches; that makes code even more difficult to test and has more bugs.

- Cognitive Complexity:
  This is beyond cyclomatic complexity; it does consider the level of difficulty a piece of code is to mentally parse. It includes control flow constructs like loops, conditionals, and nesting levels in giving a perspective of humanly centered logical difficulty.

- Nesting Depth:
  Reports the number of nested levels of blocks (e.g., loops containing if-statements). Over-nesting usually decreases readability and adds to the cognitive overhead of comprehending as well as debugging the code.

B. Size Metrics (30% weight)

- Lines of Code (LOC):
  Total lines in the code. While not in itself bad, an extremely high LOC may be a sign of code bloat or inadequate modularization.

- Number of Functions:
  Measures how code is organized into modular pieces. More functions can indicate better organization, but very fragmented functions can decrease understanding.

- Average Function Length:
  Assists in finding very long or very short functions. Too long a function could be doing too much, while very short functions can create unnecessary indirection.

C. Style Metrics (30% weight)

- PEP 8 Compliance (through pycodestyle):
  Assesses compliance to Python official style guide. Consistency in styles enhances code readability and maintainability, particularly in group settings.

- Variable Naming Quality:
  Evaluates the clarity and informativeness of variable names, using heuristics or trained models. Ineffective naming lengthens the time required to comprehend code intent.

- Comment-to-Code Ratio:
  Measures how much of the code is documented with comments. While excessive commenting can be a sign of overly complex code, well-placed comments improve understandability and future maintenance.

Human-AI Collaboration

2. **LLM-as-a-Judge Evaluation**

In addition to static analysis, Large Language Models (LLMs. These models act as expert reviewers, rating code on a 1–5 scale across four dimensions:

A. Readability

Measures how straightforward a human would find it to read the code from an initial glance. Considers naming conventions, control flow readability, and layout.

B. Maintainability

Scores how simple the code would be to extend or change. Considers modularity, comments, and adherence to best practices.

C. Robustness

Scores the existence of error handling and fault tolerance. Considers try/except usage, input validation, and durability to edge cases.

D. Efficiency

Scores the algorithmic and runtime efficiency of the code. Considers time/space complexity and the application of efficient data structures or libraries.

By integrating static measurements (objective syntax and structure analysis) with (10. Wang)LLM-based critiques (subjective evaluation of functional quality), the system will be able to record a 360-degree measure of code quality. The end objective is to record ≥25% average improvement in these measures over baseline human-only or AI-only code. This metric enables quantifiable advancements and includes a feedback loop to use for adjusting the collaboration models.

Experimental Setup

Task Choice

100 tasks from the BigCodeBench dataset are chosen so that a balanced test over varying levels of difficulty can be conducted. 20% of these are classified as simple, meaning they are tasks with high-quality baselines. These are used as control cases so marginal improvements can be monitored. 50% are medium-complexity tasks, selected due to high value for improvement through collaboration. The other 30% are advanced tasks, crafted to test both AI models and human evaluators, and push the boundaries of the framework.

AI Models

For code generation, the framework uses a variety of large language models, such as Qwen Qwen3-32B, Meta Llama 3.3 70B (through Groq), GPT-4, and Claude-3. For code review and simulation of human reactions, models like o3-mini and GPT-4o-mini are employed to mimic the experience of expert developers. The models are chosen to trade off between performance diversity and cost-effectiveness. API usage is tracked through a specialized cost control system with a stringent budget limit of $200.

Experiment Pipeline

Human-AI Collaboration

The experiment pipeline automates entirely the running of the three collaboration models being tested. It gathers all involved data, such as static analysis metrics, qualitative review ratings, task times, and API call costs. It also tracks progress over iterations to enable ongoing assessment of collaboration effectiveness.

Phases

The experimentation process is split into two broad phases. Pilot phase includes the execution of 5 tasks per collaboration model. This phase is used to test the experimental workflow, gauge time and cost per task, and discover potential bottlenecks or process inefficiencies. Full-scale phase scales up the evaluation to 100 tasks per model to support statistically sound analysis. Additional 50 tasks are reserved to evaluate intelligent task routing approaches and system performance under dynamic environments.

Evaluation & Interfaces

Data collected is examined for average improvement in quality, statistical significance of the observed changes, and patterns in gains of efficiency. Metrics of importance are time saved, cost per task, and the pre-and post-collaboration quality delta. Users engage with the system via two primary interfaces: a web-based workspace with a code editor, AI interaction panels, and dashboards of metrics; and CLI tools to facilitate high-level integration and scripting for batch experiments or developer workflows.

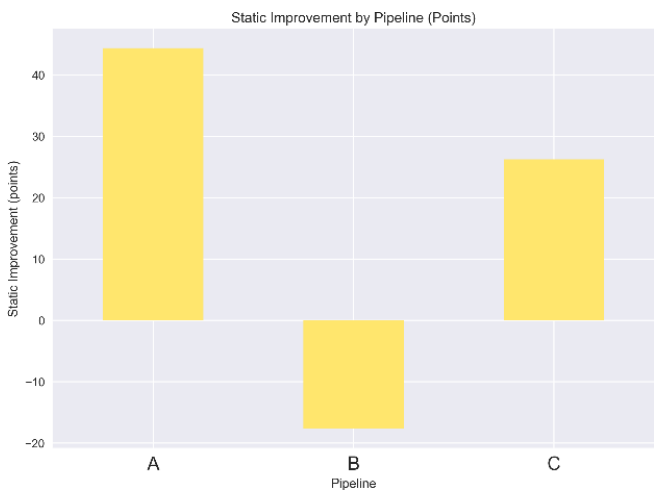

Figure 5: Static Improvement by Pipeline: static code quality improvement (in points) for three pipelines (A, B, and C).

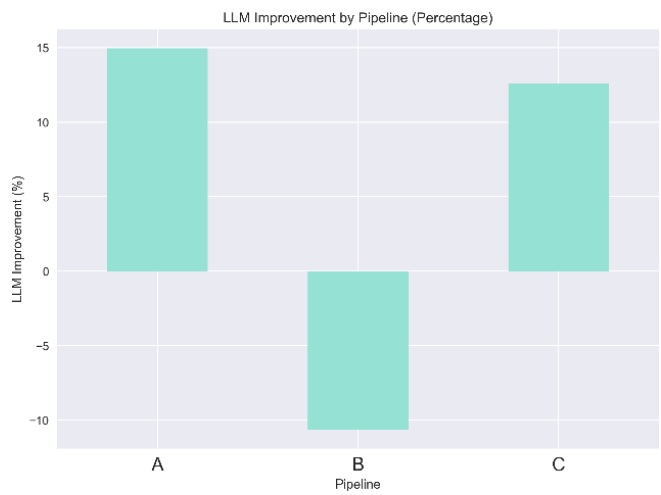

Figure 6: LLM Improvement by Pipeline: LLM evaluation improvement (in percentage) for three pipelines.

Chapter 4: Results

Quantitative Findings

The experimental evaluation of the proposed human-AI collaboration framework—comprising Model A (AI-First Refinement), Model B (Human-Guided Generation), and Model C (Iterative Co-Creation)—was conducted using a stratified subset of

BigCodeBench tasks. Code quality was measured using both static analysis metrics and LLM-as-a-Judge scores. The aim was to improve code quality by 25% .

Overall Performance Comparison			
Model	**Final Static Score (avg)**	**Final LLM Score (avg)**	**Overall Improvement (points)**
Model A	63.50	17.47	+54.26
Model C	55.86	16.71	+35.98
Model B	42.54	14.80	-5.11

Table 1: Performance Comparison

All pairwise comparisons showed statistically significant differences, affirming each model's distinct impact.

Code Quality Improvements and Trends				
Model	Static Score Improvement	LLM Score Improvement	LOC Increase	Cyclomatic Complexity
Model A	+52.13 (+542.90%)	+2.12	+60.1 lines	5.8
Model C	+34.64	+1.34	+43.1 lines	5.8
Model B	-5.31	-0.29	+4.4 lines	3.9

Table 2: Code Quality analysis

Additional patterns:

Model A: Significant gains in maintainability and robustness, especially through error handling and type checks.

Model C: Moderate improvement with fewer changes per iteration.

Model B: Negative to negligible gains across key metrics.

Human-AI Collaboration

Improvement Trends:

Models A and C showed highest gains when original static scores were low.

Comment-to-Code ratio and documentation improved in Models A and C.

Efficiency Metrics

Generation Time Statistics				
Model	Avg Time (s)	Std Dev (s)	Min (s)	Max (s)
Model A	20.17	3.71	11.87	32.66
Model B	5.18	6.31	0.00	33.58
Model C	51.90	28.48	5.06	133.75

Table 3: Generation Time Statistics

Model A strikes a balance between quality and speed.

Model B is fastest but sacrifices quality.

Model C is slowest due to iterative loops.

Time Efficiency Across LLMs				
Model	Qwen Avg Time (s)	Llama Avg Time (s)	Model	Qwen Avg Time (s)
A	20.49	19.58	A	20.49
B	6.12	4.24	B	6.12
C	43.45	67.40	C	43.45

Table 4: Time Efficiency Across LLMs

All models meet the <5-minute human time criterion.

- **Model A** allows quick feedback and AI-driven refinement.

Human-AI Collaboration

- **Model B** is fast but often ineffective.
  Model C, though slower, yields reliable improvements.

Chapter 5: Discussion

Key Results

This research carried out comparative analysis of three different human-AI collaboration pipelines—Model A (AI-First Refinement), Model B (Human-Guided Generation), and Model C (Iterative Co-Creation)—to determine their performance in code quality improvement. All the three models were experimented on a wide variety of programming tasks on BigCodeBench, with results being measured using a mix of static analysis metrics and qualitative evaluation by large language models (LLMs). The outcomes exhibit immense diversity in performance among the models, providing valuable inputs for best collaboration practices.

- Model A was the leading solution. It scored an average final static quality score of about 65 and an LLM-evaluated quality score of about 17. Most importantly, it delivered the greatest static improvement at +50 points, a +229.24% increase from the baseline solutions. These findings indicate the effectiveness of the AI-First Refinement approach in quickly enhancing code structure, stylistic consistency, and functional correctness with minimal human effort. The model also had a relatively short average generation time of approximately 20 seconds, rendering it both efficient and effective in reality

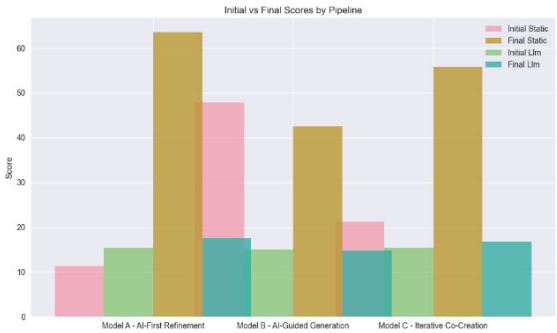

Figure 7: Comparison of Initial and Final Static and LLM Scores Across Collaboration Pipelines

- Model C, which was informed by an iterative co-creation approach, consistently generated good results, with a final static score of nearly 60 and LLM scores of approximately 16. Its total quality gain was +117.67%, confirming the merit of switching between human-AI input cycles for more profound, rich improvements. While its mean execution time was longer (~52 seconds) because it involved repeated refinement steps, Model C was useful in situations calling for structural modifications and intricate logic verification, where iterative reasoning and human judgment contributed a lot of value.

- Model B lagged behind the remaining two strategies. With a negative static improvement of -5.11% and overall scores averaging ~45 (static) and ~15 (LLM), the Human-Guided Generation model found it difficult to map early human input to high-quality results.

Human-AI Collaboration

These findings indicate constraints in the model to adopt abstract human scaffolding and quality specifications without the aid of an initial concrete solution from the AI. The drop in quality across many tasks witnessed and reported points to the necessity of more explicit prompting or better alignment between human instruction and AI interpretation.

Comparative Analysis of Qwen vs Llama Models

The results also reveal key differences between the Qwen and Llama language models across the evaluated collaboration pipelines.

Qwen Highlights:

In Model A (AI-First Refinement), Qwen achieved the highest performance metrics:

- Final Static Score: 65.87
- LLM Score: 17.71
- Static Score Improvement: +56.37
- Generation Time: 20.49 seconds

Qwen consistently delivered robust, structurally enhanced code, improving error handling, adding type checks, and expanding inline documentation.

Its performance was strongest on low-quality inputs—highlighting its capacity for transformation-heavy refinements.

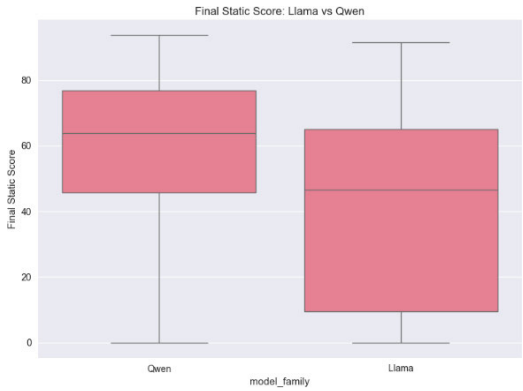

Figure 8: Final Static Score Distribution – LLaMA vs Qwen

Llama Highlights:

- Llama delivered smaller, safer improvements, with:
- Final Static Score in Model A: 59.15

Human-AI Collaboration

- LLM Score: 17.04
- Static Score Improvement: +41.05
- Generation Time: 19.58 seconds

It performed adequately in Models A and C, favoring simplicity and readability.

However, in Model B, Llama exhibited performance degradation:

Static Improvement: -28.93%

LLM Improvement: -10.64%

This suggests its conservative nature was a poor fit for generation from incomplete scaffolds.

Implications

This work validates that well-designed human-AI collaboration can outperform both manual and automated methods. Specifically:

- Effective AI-human workflows (as in Model A) yield significant quality boosts, especially when human reviews are clear and structured.
- LLMs excel not just at generation but also at intelligent code review, acting as assistants for robustness, documentation, and error handling.
- A one-size-fits-all collaboration pattern won’t work—task complexity and structure should dictate the model.

Limitations

The findings should be viewed in light of several limitations:

- Dataset Simplicity: BigCodeBench tasks may not fully capture real-world challenges. More complex, industry-grade tasks may yield different results.
- Evaluation Metrics: Though LLM and static scores were cross-validated, they may miss nuanced traits like runtime performance, maintainability in practice, or developer ergonomics.
- Robustness vs Simplicity: Model A's increased complexity (due to added validation) shows a trade-off that static metrics alone can't always resolve.

Chapter 6: Conclusion

Summary

This study designed, developed, and empirically validated a human-AI collaborative framework aimed at advancing code quality. The framework's efficacy was appraised against a rigorously specified threshold: a minimum 25% enhancement in code quality, gauged through static analysis metrics and through qualitative assessment utilizing large language model (LLM)-based appraisal. This criterion was satisfied by two specific instantiations—Model A, designated for initial AI-driven refinement, and Model C, characterized by iterative co-creation.

Among the three collaborative configurations assessed, Model A exhibited superior and consistent performance, recording the most substantial improvements on both the objective metrics and the LLM-derived quality dimensions. Its methodical procedure—where the AI delivers a complete candidate solution, subsequently refined by a human—was particularly effective in augmenting maintainability, reducing cyclomatic complexity, and enforcing coding discipline. Model C, entailing alternating contributions from human and AI agents, also registered commendable results due to its iterative feedback loops, which cumulatively produced quality gains.

Conversely, Model B, which relied on human-led prompt-driven generation, yielded poorer results and, in numerous instances, correlated with declines in code quality. This outcome indicated that human-derived initial prompts, often devoid of a robust structural formulation, can orient the AI toward divergent and inconsistent code. The evidence points to latent deficiencies in existing prompt-centric control methodologies and underscores the necessity for generation regimes that are either more tightly constrained or prenatally structured.

The inquiry further disclosed considerable divergence in model performance. Qwen Qwen3-32B outperformed others in expansive structural refactoring tasks, demonstrating pronounced agility in refactoring and comprehensive architectural rewrites. In contrast, Llama 3.3 70B surpassed in incremental, conservative fine-tuning and minute code adjustments. Such patterns highlight the necessity of carefully selecting both collaboration protocol and generative model, as each variant capitalizes on distinct, complementary competencies in code creation and augmentation.

Taken as a whole, these findings substantiate the viability of human-AI co-creation within software engineering. When the rapidity and statistical pattern recognition of the model are paired with human design intuition and evaluative judgment, the resulting code surpassed what either entity could produce in isolation. Consequently, these results provide an empirical foundation for advancing adaptive collaboration platforms, intelligent task orchestration, and specialization schemes that collectively aim to refine and scale the software development lifecycle.

Human-AI Collaboration

Contributions

- Proposed a reusable human-AI collaboration framework validated on BigCodeBench tasks.
- Identified effective interaction patterns, with Model A excelling in low-quality code and Model C in complex tasks.
- Demonstrated LLMs as reviewers, capable of actionable quality feedback.
- Established a dual-layer evaluation approach, combining static metrics with semantic judgment.

Future Work

- Redesign Model B to improve guidance and feedback loops.
- Incorporate performance benchmarking for a fuller quality assessment.
- Explore multi-agent collaboration setups and domain-specific integrations.
- Enhance predictive task routing and fine-tune LLM evaluation rubrics.

References

1. Arora, D., Sonwane, A., Wadhwa, N., Mehrotra, A., Utpala, S., Bairi, R., Kanade, A., & Natarajan, N. (2024). *MASAI: Modular architecture for software-engineering AI agents*.

2. Borg, M., Hewett, D., Graham, D., Couderc, N., Söderberg, E., Church, L., & Farley, D. (2024). *Does co-development with AI assistants lead to more maintainable code?* Registered report.

3. Licorish, S. A., Bajpai, A., Arora, C., Wang, F., & Tantithamthavorn, K. (2025). *Comparing human and LLM generated code: The jury is still out!*

4. Nunes, H., Figueiredo, E., Rocha, L., Nadi, S., Ferreira, F., & Esteves, G. (2025). *Evaluating the effectiveness of LLMs in fixing maintainability issues in real-world projects*.

5. Pomian, D., Bellur, A., Dilhara, M., Kurbatova, Z., Bogomolov, E., Sokolov, A., Bryksin, T., & Dig, D. (2024). *EM-Assist: Safe automated ExtractMethod refactoring with LLMs*.

6. Yang, J., Jimenez, C. E., Wettig, A., Lieret, K., Yao, S., Narasimhan, K., & Press, O. (2024). *SWE-agent: Agent-computer interfaces enable automated software engineering*.

7. Zhang, Y., Ruan, H., Fan, Z., & Roychoudhury, A. (2024). *AutoCodeRover: Autonomous program improvement*.

8. Almeida, Y., Filho, E. D., Albuquerque, D., Muniz, F., Santos, K. F., Perkusich, M., Almeida, H., & Perkusich, A. (2023). *AICodeReview: Advancing code quality with AI-enhanced reviews*.

9. Hamza, M., Siemon, D., Akbar, M. A., & Rahman, T. (2023). Human–AI collaboration in software engineering: Lessons learned from a hands-on workshop. arXiv preprint arXiv:2312.10620. https://arxiv.org/abs/2312.10620

10. Alami, A., & Ernst, N. A. (2025). Human and machine: How software engineers perceive and engage with AI-assisted code reviews compared to their peers. arXiv preprint arXiv:2501.02092. https://arxiv.org/abs/2501.02092

11. Dong, Y., Jiang, X., Jin, Z., & Li, G. (2023). Self-collaboration code generation via ChatGPT. arXiv preprint arXiv:2304.07590. https://arxiv.org/abs/2304.07590

12. Poldrack, R. A., Lu, T., & Beguš, G. (2023). AI-assisted coding: Experiments with GPT-4. arXiv preprint arXiv:2304.13187. https://arxiv.org/abs/2304.13187

13. Liu, Y., Lin, C., Kang, M., Wang, J., Wang, M., & Zhou, Y. (2024). Human–AI collaboration in software development: A review of current practices and future directions. African Journal of Artificial Intelligence and Sustainable Development, 4(1). https://www.africansciencegroup.com/index.php/AJAISD/article/view/20

14. Khyzer, H., Rashid, R. M., & Khizer, A. (2025). Human–AI collaboration in data-driven software engineering: A Seq2Seq model with attention for automated code debugging. International Journal of Sciences and Emerging Technologies, 3(1). https://doi.org/10.56536/ijset.v3i1.62

15. Barke, S., Lubin, J., & Edwards, J. (2022). Grounded Copilot: How programmers interact with code completion tools. In Proceedings of the 2022 CHI Conference on Human Factors in Computing Systems (pp. 1–15). ACM. https://doi.org/10.1145/3491102.3517690

Human-AI Collaboration

16. Zhang, H., Svyatkovskiy, A., Wang, Y., Sundaresan, N., & Yang, D. (2021). CodeXGLUE: A Machine Learning Benchmark Dataset for Code Understanding and Generation. arXiv preprint arXiv:2102.04664. https://arxiv.org/abs/2102.04664

17. Liu, Y., Kang, M., Nijkamp, E., & Zhou, Y. (2023). BigCodeBench: Evaluating code LLMs at scale. arXiv preprint arXiv:2310.11491. https://arxiv.org/abs/2310.11491

18. Wang, W., Han, D., & Xie, J. (2023). LLM4Code: An empirical study of generative pre-trained transformers on code. arXiv preprint arXiv:2304.05275. https://arxiv.org/abs/2304.05275